5-1 · 분산 처리와 Spark 아키텍처 · 이론 1H

분산 데이터 처리와 Spark 개요

스케일아웃 · 드라이버와 익스큐터 · 지연 평가

수도권ICT 이노베이션스퀘어
데이터 엔지니어링 & RAG 파이프라인 160H · 3기

이 시간의 목표

1시간 뒤에는

① 스케일업과 스케일아웃의 차이와 분산이 필요한 이유를 말할 수 있다

② 드라이버 · 익스큐터 · 클러스터 매니저의 역할을 구분한다

③ transformation 과 action 의 차이(지연 평가)를 안다

④ Job → Stage → Task 세 계층으로 Spark UI 를 읽는다

다음 1.5H 실습에서 PySpark 를 띄워 지연 평가를 UI 로 직접 확인합니다

스케일업과 스케일아웃 — 한 대를 키울까, 여러 대로 나눌까

스케일업 — 한 대를 더 좋게

물리적 상한이 있고 고사양일수록 급격히 비싸진다

- 방법: 메모리·CPU 를 키운다
- 장애: 한 대가 죽으면 끝
- 복잡도: 단순

VS

스케일아웃 — 여러 대로 나눔

대수를 늘리면 되고 한 대가 죽어도 나머지가 잇는다

- 비용: 평범한 장비 여럿
- 장애: 한 대가 죽어도 나머지가
- 복잡도: 나누고 모으는 일이 생긴다

드라이버는 주방장, 익스큐터는 요리사



4-4 의 Trino 코디네이터·워커와 같은 구조 — 분산 시스템 대부분이 이 모양

collect() 는 모든 데이터를 드라이버 한 곳으로 끌어온다 — 익스큐터가 아무리 많아도 드라이버 메모리를 넘으면 죽는다

역할 셋 — 드라이버 · 익스큐터 · 클러스터 매니저

1. **드라이버**
   내가 쓴 코드가 도는 곳 — 실행 계획을 세우고 결과를 모은다
2. **익스큐터**
   실제 계산을 하는 프로세스 — 여러 개가 병렬로 돈다
3. **클러스터 매니저**
   익스큐터를 몇 개 띄울지 정하는 관리자 — Standalone · YARN · Kubernetes

이번 주에 만드는 것

5-2 에서

Standalone

> 「한 대로 되면 한 대로 하라」— 분산은 나누고 모으는 비용이 붙는다. 3-3 DuckDB 로 감당되는 수십 GB 라면 그쪽이 훨씬 간단

RDD 와 DataFrame — 실무는 거의 항상 DataFrame

	RDD	DataFrame
모양	조각난 객체 목록	컬럼과 스키마가 있는 표
최적화	없음 — 쓴 대로 실행	Catalyst 가 계획을 다시 짬
API	map · filter · reduce	select · filter · groupBy
SQL	안 됨	됨 (SparkSQL)
언제	아주 특수한 저수준 제어	거의 모든 경우

3주차 Pandas DataFrame 과 이름도 개념도 같다 — 다른 점은 「여러 대에 나뉘어 있다」는 것뿐

4. 지연 평가

계획만 쌓다가 action 에서 한 번에

transformation — 계획만

「할 일 목록」에 적어 두기만 하고 실행하지 않는다

- select · filter · where
- withColumn · drop
- groupBy … agg · join · union
- orderBy · distinct

VS

action — 실행 시작

호출되는 순간 쌓인 목록을 최적화해 한 번에 실행

- count
- show · head · take
- collect
- write · foreach

왜 계획만 쌓는가 — 그리고 「안 되는 것 같다」의 정체

1. **전체 흐름을 보고 계획**
   쌓인 할 일 목록 전체를 보고 나서 가장 효율적인 실행 계획(DAG)을 짠다

2. **「5행만 보여 줘」**
   100만 행 읽기 → 컬럼 계산 → 5행 보기라면, 5행만 계산하고 끝낼 수 있다

3. **화면에 아무것도 안 나온다**
   transformation 만 실행하면 출력이 없다 — 오류가 아니라 정상, action 을 붙여야 돈다

Spark UI 를 읽는 세 계층

action	Job	Stage	Task
count · show · write 를 부른다	action 을 불러야 생긴다 — 하나가 여럿도	셔플을 경계로 나뉜다	파티션 하나가 Task 하나

실습에서 count() 한 번에 Job 둘, show(5) 한 번에 셋 · Stage 가 여럿이면 셔플이 있었다 · 파티션 4개면 Task 4개

느린 쿼리는 Stage 별 시간을 보고, 오래 걸린 Stage 의 Task 분포를 본다 — 5-5 에서 본격적으로

정리

이 1시간에서 꼭 가져갈 것

1. 한 대로 되면 한 대로 한다 — 분산은 나누고 모으는 비용이 붙는다
2. 드라이버는 계획, 익스큐터는 계산 — collect() 는 드라이버로 다 끌어온다
3. 실무에서는 DataFrame — 3주차 Pandas 감각이 그대로 이어진다
4. transformation 은 계획만 쌓고, action 이 실행을 시작한다
5. Job(action) → Stage(셔플 경계) → Task(파티션) 으로 UI 를 읽는다

5-1 · 이론 1H 마무리

다음은 단일 노드 Spark 기동과 PySpark 첫 실행 (1.5H)

컨테이너로 PySpark 를 띄우고, 지연 평가를 Spark UI(localhost:4040)로 직접 확인합니다

- ✓ **apache/spark:3.5.3 컨테이너로 pyspark 셸**
- ✓ **spark.version · master 가 local[*] 인지**
- ✓ **transformation 뒤 Completed Jobs 가 없는지**
- ✓ **count() 로 Job 둘 · show(5) 로 셋 · explain()**

준비할 것

Docker 실행 · 4주차 컨테이너 멈추기(stop)